SMART CITY · SOFTWARE ENGINEERING

SmartWaste

Smart City Waste & Recycling Manager
Technical Documentation

ARCHITECTURE · DATA MODEL · API CONTRACTS · SERVICE WORKFLOWS

Prepared from the supplied project presentation
37-page source deck · 13 November 2026

Document guide

This technical document expands the supplied presentation into a structured engineering reference. It preserves the architecture, service boundaries, data fields, API summaries and event flows shown in the source. Where the presentation is incomplete or internally inconsistent, the document flags the point for team confirmation instead of assuming an implementation.

Reading the document
Sections 1–7 describe the platform-wide design. Sections 8–12 detail the five microservices. Sections 13–15 consolidate workflows, deployment and verification. Section 16 lists unresolved decisions.

Source boundary
No implementation repository, code, OpenAPI specification, database migration, deployment manifest or test report was supplied with the presentation. Accordingly, this document does not claim that every proposed component is already implemented or verified.

CHAPTER 01

Executive overview

SmartWaste is a centralized smart-city waste and recycling management platform connecting citizens, garbage trucks, smart recycling bins and city administrators. Its central objective is to close the information gap between what happens in the street, what citizens report and what collection teams know.

The presentation identifies three operational problems: fixed collection schedules do not reflect real bin fill levels; citizen reporting is slow and scattered; and the absence of timely feedback weakens incident response and recycling participation.

> **The proposed response is a five-service platform organized around monitoring, planning, citizen engagement and notifications.**

Expected benefits

- Cleaner public spaces
- Fewer unnecessary collection trips
- Better recycling participation
- Faster incident resolution
- Better use of city resources

CHAPTER 02

Problem, objectives and scope

Problem statement

- Trucks may visit half-empty bins while full bins wait because collection follows fixed schedules rather than current demand.
- Citizens lack a simple way to report a full or broken bin; reports can remain unresolved for days.
- Teams are not warned in time, and recycling contributions are not rewarded promptly.

Project objectives

1. Monitor bin fill levels and locations.
2. Use current bin information to support collection planning.
3. Allow citizens to submit incident reports with a photo.
4. Track report status through resolution or rejection.
5. Notify the appropriate role when important events occur.
6. Support eco-points and rewards to encourage participation.

Actors and scope

Actor	Capabilities described in the presentation
Citizen	Register and log in; manage profile; find bins; report incidents with a photo; view eco-points; receive notifications.
Driver	View assigned pickup routes; update route status; view bins and fill levels; receive alerts.
Administrator	Manage users and points; manage bins and bin types; monitor fill levels; manage trucks and routes; review reports; monitor alerts.

Documentation note
The presentation defines the actor-facing scope. Detailed user stories, acceptance criteria, service-level targets and explicit out-of-scope items are not supplied.

CHAPTER 03

System architecture

The system is decomposed into five independently responsible microservices. The split is by business capability, not by database table or screen.

Service	Business responsibility	Owned data	Database
MS1 — Identity & Rewards	Accounts, roles, profiles, eco-points and reward history	User, RewardHistory	PostgreSQL in architecture overview; MySQL on MS1 work-plan slide — decision required
MS2 — Smart Bin Asset	Bin locations, capacity, fill level and bin types	SmartBin, BinType	MongoDB
MS3 — Fleet & Routing	Trucks, pickup routes, route stops and collection status	GarbageTruck/Truck, PickupRoute/Route, RouteStop	PostgreSQL
MS4 — Incident Reporting	Citizen reports, photos and report status	IncidentReport	MongoDB
MS5 — Real-Time Alerts	Consumes events, creates alerts and targets users by role	Alert	H2 in-memory for the presented setup; PostgreSQL is identified as a production migration path

Platform building blocks

Component	Role in the platform
API Gateway	Single public entry point; routes requests and verifies JWTs before forwarding them.
Keycloak	Identity provider; handles sign-in and issues signed JWTs containing user roles.
Eureka Server	Service registry; services register by name and discover one another without fixed addresses.
Config Server	Provides service configuration at startup from a Git-based config-repo.
RabbitMQ	Carries asynchronous domain events through the smartwaste.events exchange.
Docker Compose	Starts services as containers on the private smartwaste-net network.

Documentation note
The presentation names Angular / React and Flutter as frontend technologies, but does not select one definitive web framework or provide application-level frontend architecture.

CHAPTER 04

Technology stack

Area	Technologies stated
Backend	Java 17, Spring Boot 3; REST APIs; one Spring Boot project per microservice.
Spring Cloud	Gateway, Eureka, Config Server and OpenFeign.
Security	Keycloak, OAuth2 and JWT.
Messaging	RabbitMQ; exchange smartwaste.events.
Databases	PostgreSQL, MongoDB and H2.
Frontend and delivery	Angular / React, Flutter, Docker and Git.

The presentation describes the stack at platform level. Exact dependency versions, build plugin versions, environment variables, container image tags and deployment hosting details are not included.

CHAPTER 05

Data ownership and persistence

Each microservice owns its database. There are no shared tables and no cross-service foreign keys. When a service needs another service's information, it calls that service's API or reacts to an event.

Service data models

Service / entity	Fields shown in the presentation
MS1 User	id: Long; username: String; email: String (unique); role: String; pointsBalance: Integer.
MS1 RewardHistory	id: Long; points: Integer; reason: String; awardedAt: LocalDateTime. One User has zero or more reward-history entries.
MS2 BinType	id: String; name: String (unique); recyclable: boolean.
MS2 SmartBin	id: String; latitude: Double; longitude: Double; capacity: Double; fillLevel: Double; status: String. Each bin belongs to one bin type.
MS3 Truck	id: Long; plateNumber: String; capacityKg: double; status: TruckStatus.
MS3 Route	id: Long; date: LocalDate; status: RouteStatus; truckId: Long; driverId: String; totalDistanceKm: double; durationMin: int; estimated: boolean.
MS3 RouteStop	id: Long; routeId: Long; binId: String; latitude: double; longitude: double; sequenceOrder: int; status: StopStatus; collectedAt: LocalDateTime.
MS4 IncidentReport	id: String; title: String; description: String; imagePath: String; createdAt: LocalDateTime; status: ReportStatus; userId: Long; binId: String.
MS5 Alert	id: Long (auto-increment); eventId: UUID (unique idempotency token); sourceId: String; message: String; severity: AlertSeverity; targetRole: String; createdAt: LocalDateTime. Operations shown: isCritical(), acknowledge().

Enumerations

Enumeration	Values
TruckStatus	AVAILABLE, ON_ROUTE, MAINTENANCE
RouteStatus	PLANNED, IN_PROGRESS, COMPLETED, CANCELLED
StopStatus	PENDING, COLLECTED, SKIPPED
ReportStatus	PENDING, RESOLVED, REJECTED
AlertSeverity	INFO, WARNING, CRITICAL

Documentation note
The MS3 class diagram uses the class name Route while the service overview calls the owned object PickupRoute. This document uses Route for the class diagram and notes the naming variation.

CHAPTER 06

Inter-service communication

The platform uses two communication modes. Synchronous REST/OpenFeign calls are used when a caller needs an immediate answer. Asynchronous RabbitMQ events are used to announce completed actions without making the publisher wait for consumers.

#	From → To	Mode	Purpose / payload
1	MS4 → MS1	Synchronous · OpenFeign	GET /api/users/{id} validates that the reporting user exists before a report is accepted.
2	MS4 → MS2	Synchronous · OpenFeign	GET /api/bins/{id} validates that the referenced bin exists.
3	MS3 → MS2	Synchronous · OpenFeign	GET /api/bins/{id} retrieves bin coordinates and capacity for route planning.
4	MS2 → MS3 and MS5	Asynchronous · RabbitMQ	BinFullEvent announces binId, location and fill level when the threshold is reached.
5	MS4 → MS5	Asynchronous · RabbitMQ	IncidentReportedEvent announces a new report using report/incident ID, bin, title and date.

Additional event flows defined in service-specific slides

- MS3 publishes RouteCompletedEvent. MS2 consumes it and resets the fill level of collected bins.
- MS4 is described as publishing IncidentResolvedEvent when a report is resolved. MS1 consumes it, creates RewardHistory and increments pointsBalance.
- MS5 consumes BinFullEvent and IncidentReportedEvent.

Reliability behavior described

- For MS4 validation calls, a timeout/fallback may allow the report to be saved as PENDING when MS1 or MS2 is unavailable; retry is described for later.
- For MS3, if MS1 or MS2 is down, timeout and circuit breaker behavior is described; new route creation is refused with HTTP 503, while saved routes retain stored coordinates.
- If the external routing API is down, MS3 uses a straight-line estimate and marks estimated=true.
- If RabbitMQ is down, the presentation says events are retried or wait in the queue. If MS5 is down, its queued event waits until the consumer restarts.

Documentation note
The deck does not specify exact retry intervals, retry limits, dead-letter routing policy beyond the MS5 DLX name, timeout durations or circuit-breaker thresholds.

CHAPTER 07

Security model

The presentation defines a login-once flow based on Keycloak-issued JWTs and API Gateway validation.

1. The user signs in through Keycloak from the web or mobile application.
2. Keycloak returns a signed JWT containing the user's role.
3. The client sends API requests to the API Gateway with the JWT.
4. The gateway verifies the token signature using Keycloak's public keys.
5. If valid, the gateway forwards the request to the target microservice.

Rule	Expected behavior
Missing or invalid token	401 Unauthorized.
Authenticated user with insufficient role	403 Forbidden.
Public exposure	Only the API Gateway is public; microservices remain on the private network.
Role examples	CITIZEN, DRIVER and ADMINISTRATOR are the platform actors; some service contracts use OPERATOR as a recipient role.

Documentation note
The presentation does not define token lifetimes, refresh-token policy, password policy, fine-grained authorization matrix, CORS policy, secrets management or encryption-at-rest configuration.

CHAPTER 08

MS1 — Identity & Rewards

MS1 manages registration, login, user profiles, roles, eco-points and reward history. It owns User and RewardHistory.

Work plan

1. Create the User and RewardHistory data model.
2. Implement registration, login and role handling.
3. Implement profile and reward-history endpoints.
4. Consume IncidentResolvedEvent and award points.
5. Add unit tests and document the API contract.

Relationships and integration

A User has zero or more RewardHistory entries. Other services store a plain userId rather than a foreign key. MS4 calls MS1 synchronously to validate a userId. MS1 consumes IncidentResolvedEvent from MS4 to award eco-points.

API contract

Method	Endpoint	Codes	Purpose
POST	/api/auth/register	201, 400, 409	Create an account.
POST	/api/auth/login	200, 401	Authenticate and return a token.
GET	/api/users/{id}	200, 404	Read profile and balance.
PUT	/api/users/{id}	200, 400, 404	Update profile.
GET	/api/users/{id}/rewards	200, 404	List RewardHistory.
POST	/api/users/{id}/points	200, 400, 404	Award points (internal).

Reward processing rule: create a RewardHistory entry and increment pointsBalance in one transaction. Duplicate events are ignored by eventId.

Documentation note
Database discrepancy: the platform overview lists PostgreSQL for MS1, while the MS1 scope/work-plan slide lists MySQL. The implementation choice must be confirmed before final submission.

CHAPTER 09

MS2 — Smart Bin Asset

MS2 manages bin location, capacity, fill level and bin types. It owns SmartBin and BinType in MongoDB.

Work plan

1. Create SmartBin and BinType collections.
2. Implement bin and bin-type CRUD endpoints.
3. Implement validated fill-level updates.
4. Publish BinFullEvent when the configured threshold is reached.
5. Add unit tests and document the API contract.

API contract

Method	Endpoint	Codes	Purpose
POST	/api/bins	201, 400, 404	Register a bin.
GET	/api/bins	200	List bins; filter by type or status.
GET	/api/bins/{id}	200, 404	Read one bin.
PATCH	/api/bins/{id}/fill-level	200, 400, 404	Update fill level; may emit an event.
GET	/api/bin-types	200	List bin types.
POST	/api/bin-types	201, 400, 409	Create a bin type.

Event contract

```
{ "eventId": "uuid", "binId": "…", "fillLevel": 90, "occurredAt": "…" }
```

The event is published when fillLevel reaches the threshold. The platform-level design states that MS3 and MS5 consume BinFullEvent. The MS5 design identifies the critical-alert trigger as fillLevel ≥ 90%.

Documentation note
The deck does not define the complete SmartBin status enumeration, exact fill-level validation range, sensor protocol or the threshold configuration mechanism.

CHAPTER 10

MS3 — Fleet & Routing

MS3 manages garbage trucks, daily pickup routes, ordered route stops and collection progress. Administrators plan routes; drivers follow assigned routes and mark stops as collected.

Service profile

Property	Value
Spring Boot port	8083
Eureka service name	FLEET-ROUTING-SERVICE
Database	PostgreSQL (fleet_db)
Gateway routes	/api/trucks and /api/routes

Core workflows

1. Manage trucks using plate number, capacity and status (AVAILABLE, ON_ROUTE or MAINTENANCE).
2. Plan a daily route using a truck, a driver, a date and an ordered list of bins; obtain distance information from a routing API.
3. Track collection by marking individual stops as COLLECTED. When the route is completed, publish RouteCompletedEvent so MS2 can reset collected-bin fill levels.

Data model

Entity	Attributes
Truck	id: Long; plateNumber: String; capacityKg: double; status: TruckStatus.
Route	id: Long; date: LocalDate; status: RouteStatus; truckId: Long; driverId: String; totalDistanceKm: double; durationMin: int; estimated: boolean.
RouteStop	id: Long; routeId: Long; binId: String; latitude: double; longitude: double; sequenceOrder: int; status: StopStatus; collectedAt: LocalDateTime.

Relationship constraints shown: one Truck can be associated with zero or more Routes; one Route contains one or more RouteStops. The slide notes that relational transactions prevent a truck from having two routes on one day.

Service layers and clients

- RouteController exposes the REST API.
- RouteService contains business rules.
- RouteRepository persists route data in PostgreSQL.
- BinClient uses OpenFeign to call MS2.
- UserClient uses OpenFeign to call MS1.
- RoutingApiClient calls an external routing API over REST.
- RouteEventPublisher publishes events through RabbitMQ.

Interactions and failure behavior

- MS3 calls MS1 with GET /api/users/{id} to check the driver.
- MS3 calls MS2 with GET /api/bins/{id} to obtain bin coordinates.
- The routing API provides distance and duration.
- If MS1 or MS2 is unavailable, the described timeout/circuit-breaker path refuses route creation with 503; previously saved routes retain stored coordinates.
- If the routing API is unavailable, a straight-line estimate is used and estimated=true is stored.
- MS3 publishes RouteCompletedEvent to exchange smartwaste.events; MS2 consumes it and resets collected bins.

API contract

Method	Endpoint	Purpose	Codes
POST	/api/routes	Create a route.	201, 400, 404, 409, 503
GET	/api/routes	List routes by date/status.	200
GET	/api/routes/{id}	Get a route with stops.	200, 404
PATCH	/api/routes/{id}/status	Start, complete or cancel a route.	200, 400, 403, 404
PATCH	/api/routes/{id}/stops/{stopId}	Mark a stop collected.	200, 400, 404
POST	/api/routes/{id}/optimize	Reorder stops by distance.	200, 404, 503
CRUD	/api/trucks/{id}	Manage trucks.	200, 201, 204, 400, 404, 409

All routes require a JWT (401 if missing). Truck changes are administrator-only (403). Drivers read their assigned routes and mark stops.

RouteCompletedEvent

```
{ "eventId": "uuid", "routeId": "…", "truckId": "…", "completedAt": "…", "collectedBinIds": ["…"] }
```

Documentation note
The routing API provider, optimization algorithm, distance metric, vehicle-capacity constraints, driver scheduling rules and exact truck CRUD sub-endpoints are not specified in the presentation.

CHAPTER 11

MS4 — Incident Reporting

MS4 receives citizen reports about problems such as a full bin, a broken bin or rubbish on the ground. Administrators follow each report until it is resolved or rejected.

Service profile

Property	Value
Spring Boot port	8084
Eureka service name	INCIDENT
Database	MongoDB (incident_db)
Gateway route	/api/incidents

Data model

Field	Type / meaning
id	String
title	String
description	String
imagePath	String
createdAt	LocalDateTime
status	ReportStatus: PENDING, RESOLVED or REJECTED
userId	Long; plain identifier owned by MS1
binId	String; plain identifier owned by MS2

One report is stored as one MongoDB document. The report stores userId and binId as identifiers only; it does not create cross-service foreign keys or duplicate the full user/bin records.

Internal layers

- IncidentController exposes the REST API.
- IncidentService contains business rules.
- IncidentRepository persists documents in MongoDB.
- BinClient validates bins through OpenFeign to MS2.
- IncidentEventPublisher publishes RabbitMQ events.

Interactions and resilience

- MS4 is called through the API Gateway at /api/incidents.
- Before accepting a report, MS4 calls MS2 GET /api/bins/{id} and MS1 GET /api/users/{id}.
- MS4 publishes IncidentReportedEvent so MS5 can notify administrators.

- If MS1 or MS2 is unavailable, timeout/circuit-breaker behavior is described; the report may be saved as PENDING and the check retried later.
- If RabbitMQ is unavailable, the report is still saved and event publication is retried. If MS5 is unavailable, the event waits in the queue until MS5 restarts.

API contract

Method	Endpoint	Purpose	Codes
POST	/api/incidents	Create a report with photo.	201, 400, 404
GET	/api/incidents	List reports, optionally by status.	200
GET	/api/incidents/{id}	Retrieve one report.	200, 404
PATCH	/api/incidents/{id}/status	Administrator changes report status.	200, 400, 403, 404
DELETE	/api/incidents/{id}	Delete a report.	204, 404

IncidentReportedEvent

```
{ "eventId": "uuid", "incidentId": "…", "binId": "…", "title": "…", "createdAt": "…" }
```

All MS4 routes require a JWT. PATCH and DELETE are administrator-only. MS5 consumes IncidentReportedEvent and creates an alert.

Documentation note
The presentation refers to photo upload and imagePath but does not specify file storage (local, object storage or database), file-size limits, accepted formats or retention rules. The exact IncidentResolvedEvent payload is also not supplied.

CHAPTER 12

MS5 — Real-Time Alerts

MS5 listens for domain events, creates alerts and notifies the appropriate user role. It is described as the platform's event-driven, real-time service.

Service profile and persistence

Property	Value
Spring Boot port	8085
Messaging	RabbitMQ; exchange smartwaste.events
Database	Dedicated H2 in-memory database (alerts_db) in the presented setup
Production path	The slide describes switching to PostgreSQL through application.properties.

Alert data model

Field / operation	Definition
id	Long; primary key, auto-increment.
eventId	UUID; unique idempotency token.
sourceId	String; source identifier, e.g. binId or reportId.
message	String; human-readable description.
severity	AlertSeverity: INFO, WARNING or CRITICAL.
targetRole	String; recipient role, examples DRIVER and OPERATOR.
createdAt	LocalDateTime; alert trigger timestamp.
isCritical()	Returns whether the alert is critical.
acknowledge()	Acknowledges the alert.

Severity rules

Severity	Meaning / trigger
INFO	Routine status update or scheduled reminder.
WARNING	Incident reported or maintenance needed.
CRITICAL	Bin fill level ≥ 90%; urgent pickup required.

Event processing

- MS2 publishes BinFullEvent when fillLevel ≥ 90%; the event key is bin.full.
- MS4 publishes IncidentReportedEvent; the event key is incident.reported.
- RabbitMQ exchange: smartwaste.events (topic).
- Durable queues: alerts.bin-full and alerts.incident-reported.

- Dead-letter exchange: smartwaste.events.dlx for poison messages.
- MS5 checks eventId for idempotency before creating an alert.
- A critical bin-full alert targets DRIVER; a warning incident-reported alert targets OPERATOR.

API contract

Method	Endpoint	Purpose	Success / error
GET	/api/alerts	List active alerts; filters ?severity= and ?role=.	200 List; 500
GET	/api/alerts/{id}	Retrieve alert details by ID.	200 AlertDTO; 404
GET	/api/alerts/severity/{level}	Filter alerts by INFO, WARNING or CRITICAL.	200 List; 400
DELETE	/api/alerts/{id}	Acknowledge/dismiss an active alert.	204; 404

Documentation note
The presentation describes role-targeted alerts and REST endpoints, but does not specify push delivery technology (WebSocket, mobile push, email, etc.), read/unread state, alert retention or the exact relationship between acknowledge() and DELETE.

CHAPTER 13

End-to-end business workflows

A. Bin reaches the critical fill level

1. A bin's fill level is updated in MS2.
2. When the threshold is reached (≥ 90% in the MS5 design), MS2 publishes BinFullEvent.
3. RabbitMQ routes the event to the fleet/routing and alerts consumers.
4. MS5 checks eventId to prevent duplicate alert creation and creates a CRITICAL alert targeted at DRIVER.
5. MS3 can use the bin data to support pickup planning.

B. Citizen submits an incident

1. Citizen submits title, description, photo and the concerned bin through MS4.
2. MS4 validates the user with MS1 and the bin with MS2 using synchronous OpenFeign calls.
3. MS4 stores the report with PENDING status and publishes IncidentReportedEvent.
4. MS5 consumes the event and creates a WARNING alert targeted at OPERATOR.
5. An administrator reviews the report and changes its status to RESOLVED or REJECTED.

C. Incident resolution and eco-points

1. MS4 emits IncidentResolvedEvent when an incident is resolved (event referenced in MS1 design).
2. MS1 consumes the event.
3. MS1 creates a RewardHistory entry and increments the user's pointsBalance in one transaction.
4. MS1 ignores duplicate events using eventId.

D. Route planning and collection

1. Administrator selects a truck, driver, date and bins for a daily route.
2. MS3 validates the driver through MS1 and retrieves bin coordinates through MS2.
3. MS3 requests distance and duration from an external routing API; if unavailable, it uses a straight-line estimate and marks estimated=true.
4. MS3 stores the route and ordered stops.
5. Driver views the assigned route and updates route/stop status.
6. When the route is completed, MS3 publishes RouteCompletedEvent; MS2 consumes it and resets fill levels for collected bins.

CHAPTER 14

Deployment and runtime topology

The presentation proposes Docker Compose to start each service in its own container on a private network named smartwaste-net. The API Gateway is the only public entry point.

Component	Port / identifier stated
API Gateway	8080 (shown in MS5 API slide)
MS3 Fleet & Routing	8083; Eureka name FLEET-ROUTING-SERVICE
MS4 Incident Reporting	8084; Eureka name INCIDENT
MS5 Real-Time Alerts	8085
RabbitMQ broker	5672
MS1 / MS2	Ports not specified in the supplied slides.

Config Server retrieves service configuration at startup from a Git repository named config-repo. Eureka provides service discovery. The presentation does not provide a complete deployment manifest, health-check configuration, resource limits, production host, TLS termination or backup strategy.

CHAPTER 15

Testing and quality strategy

The service work plans explicitly include unit tests and API documentation for MS1 and MS2; equivalent testing detail is not fully enumerated for every service. A complete verification plan should cover the behaviors already specified in the design.

Test level	Recommended verification derived from stated contracts
Unit tests	Validate service rules, status transitions, threshold handling, route constraints and reward transaction behavior.
API tests	Verify endpoint methods, request validation and documented success/error codes.
Integration tests	Verify OpenFeign calls between MS4/MS1/MS2 and MS3/MS1/MS2.
Messaging tests	Verify event publication/consumption, event payloads, idempotency and queue recovery.
Security tests	Verify missing JWT returns 401 and role-restricted actions return 403.
Resilience tests	Exercise dependency timeouts, circuit breakers, routing API fallback and RabbitMQ interruption.

Documentation note
The test levels above are a proposed verification checklist based on the presentation's behavior; no test framework, coverage target or completed test results are provided in the source deck.

CHAPTER 16

Open decisions and source consistency

The following points should be confirmed by the team before treating this document as a final implementation specification.

Topic	What the presentation says	Confirmation needed
MS1 database	PostgreSQL in the platform service matrix; MySQL in the MS1 scope/work-plan slide.	Which database is actually used? Update the architecture matrix and service implementation accordingly.
MS5 database	H2 in the service matrix and design slide; PostgreSQL is described as a production migration path.	Is H2 development-only, and is PostgreSQL the deployment target?
MS1 and MS2 ports	Not shown in the supplied deck.	Record actual server.port values and Eureka service IDs.
MS3 routing provider	An external routing API returns distance and duration.	Name the provider, API contract, key management and fallback calculation.
Route optimization	API endpoint says reorder stops by distance.	Specify algorithm, constraints and whether optimization is nearest-neighbor, shortest-path or another method.
MS4 photo storage	IncidentReport contains imagePath; photo upload is supported.	Specify storage location, access control, size/type limits and retention.
Event contracts	Several event names and partial payloads are shown.	Publish a canonical schema for each event, including IncidentResolvedEvent and RouteCompletedEvent.
MS5 delivery channel	Alerts are created and targeted by role.	Specify how users receive them (in-app polling, WebSocket, push, email, etc.).
Event reliability	Retries, durable queues, idempotency and a DLX are mentioned.	Specify retry/backoff, acknowledgements, dead-letter handling and replay procedures.
Identity naming	Actor is called Administrator; some alert targeting uses OPERATOR.	Confirm canonical role names and authorization matrix.
API details	Endpoint and status-code summaries are shown.	Add request/response schemas, pagination, filters, validation rules and examples.

This document preserves the presentation's stated design and marks missing implementation details as open decisions rather than inventing them.

CHAPTER 17

Glossary

Term	Meaning
API Gateway	Single entry point that routes client requests and validates JWTs.
Eureka	Service registry and discovery mechanism.
OpenFeign	Declarative HTTP client used for synchronous service-to-service calls.
JWT	Signed token carrying identity and role claims.
RabbitMQ	Message broker used to deliver asynchronous events.
Event-driven architecture	Services react to events published by other services without requiring a synchronous response.
Idempotency	Processing the same event more than once does not create duplicate effects.
Circuit breaker	Resilience mechanism that stops repeated calls to an unavailable dependency.
DLX	Dead-letter exchange used to route messages that cannot be processed normally.
CRUD	Create, read, update and delete operations.

CHAPTER 18

Conclusion

SmartWaste proposes a microservices-based platform that connects real-time bin information, collection planning, citizen incident reporting, rewards and role-targeted alerts. Clear service ownership and separate databases support independent responsibilities, while REST/OpenFeign and RabbitMQ provide synchronous validation and asynchronous event propagation.

The design is sufficiently detailed to describe the platform boundaries, principal entities, core API contracts and key event flows. Before implementation or formal acceptance, the team should resolve the consistency items in Section 16 and complete the missing operational specifications.